

LAB 1 - BLE MAC Address Anonymisation

Guida completa con AirPods Pro 2 su MacBook Air M2 (macOS Sequoia 15.x)

Panoramica del laboratorio

Obiettivo

Dimostrare il meccanismo di **anonimizzazione degli indirizzi MAC BLE** tramite i Resolvable Private Address (RPA), osservandolo da due prospettive opposte:

- **Alice (trusted):** ha il pairing con le AirPods → vede l'identità reale (PMAC)
- **Trudy (passiva):** nessun pairing → vede solo RPA che ruotano ogni ~15 min

Concetti chiave

Termine	Significato
PMAC	Public MAC address — l'identità reale e stabile delle AirPods
RPA	Resolvable Private Address — indirizzo temporaneo cifrato con l'IRK
IRK	Identity Resolving Key — chiave scambiata durante il bonding, permette di risolvere un RPA nel PMAC
Bonding	Fase dopo il pairing in cui i dispositivi si scambiano e memorizzano le chiavi (LTK, IRK, CSRK)

Come funziona l'RPA

Le AirPods Pro 2 usano BLE 5.x con Secure Connections. Ogni ~15 minuti generano un nuovo RPA:

```
prand      = 22 bit casuali
hash       = ah(IRK, prand) → 24 bit
RPA        = hash || prand || "10" (i 2 bit finali identificano il tipo)
```

Chi possiede l'IRK (cioè il MacBook accoppiato) può ricalcolare l'hash e verificare l'identità. Chi non lo ha (Trudy) vede solo MAC sempre diversi.

Host Controller Interface: l'interfaccia standard dello stack Bluetooth che separa due parti distinte dell'hardware/software:

Host → il sistema operativo e gli strati software superiori (L2CAP, SMP, GATT, ecc.)

Controller → il chip fisico BLE che gestisce la radio, il Link Layer e la PHY

Fase 0 - Installazione degli strumenti

0.1 PacketLogger (tool di cattura BT su macOS)

PacketLogger è il tool Apple per catturare traffico **HCI**. Non richiede Xcode completo (~15 GB).

1. Vai su <https://developer.apple.com/download/all/>
2. Accedi con il tuo Apple ID (gratuito)
3. Cerca "Additional Tools for Xcode 16"
4. Scarica il `.dmg` (~200 MB)
5. Apri il `.dmg` → cartella `Hardware` → trascina **PacketLogger.app** in `/Applications`

0.2 Logging Profile (necessario per catturare su macOS)

Senza questo profilo PacketLogger non cattura nulla.

1. Vai su <https://developer.apple.com/bug-reporting/profiles-and-logs/>
2. Scarica il profilo **Bluetooth** (file `.mobileconfig`)
3. Aprilo — macOS chiederà di installarlo
4. Vai in `Impostazioni di Sistema → Generali → Gestione VPN e dispositivi`
5. Seleziona il profilo → clicca **Installa**
6. Riavvia il Mac

Nota: Ricordati di rimuovere il profilo al termine del lab (riduce le performance del BT in uso normale).

0.3 Wireshark

1. Scarica da <https://www.wireshark.org>
2. Installa normalmente (versione macOS ARM per M2)
3. PacketLogger salva in formato `.btsnoop` o `.pkg` — Wireshark li legge entrambi

Fase 1 - Traccia ALICE (dispositivo trusted, pairing attivo)

Scenario

Il MacBook è **già accoppiato** con le AirPods Pro 2. Il BT stack risolve automaticamente gli RPA nell'identità reale tramite l'IRK memorizzato durante il bonding precedente.

1.1 Verifica che le AirPods siano accoppiate

Impostazioni di Sistema → Bluetooth
→ Le AirPods Pro devono apparire nella lista "I miei dispositivi"

1.2 Avvia la cattura con PacketLogger

1. Apri **PacketLogger.app**
2. Vai su `File → New Bluetooth Logger` (oppure $\text{⌘}N$)
3. Clicca **Start** ($\blacktriangleright$)
4. La cattura è iniziata sull'interfaccia HCI interna del Mac

1.3 Genera traffico BLE

- Metti le AirPods nelle orecchie oppure aprile dal case vicino al Mac
- Aspetta che il Mac le rilevi e si connetta (comparirà il nome nella menu bar)
- Lascia girare la cattura per almeno **5 minuti**
- Puoi anche disconnetterle e riconnetterle per catturare la fase di connection

1.4 Salva la traccia Alice

1. PacketLogger: `File → Save As`
2. Salva come `Sxxxxxx_AirPodsPro2_Alice.btsnoop`
3. Questo formato è direttamente apribile con Wireshark

1.5 Analisi in Wireshark – Traccia Alice

Apri il file in Wireshark

Step 1 — Trova l'indirizzo identity (PMAC)

Rimuovi il rumore degli advertising report:

```
!(bthci_evt.code == 0x3e)
```

Cerca i pacchetti di connessione — l'indirizzo lì presente è il PMAC risolto dall'IRK. Annotalo, sarà nella forma `XX:XX:XX:XX:XX:XX`.

Step 2 — Filtra tutti i pacchetti del dispositivo

```
bthci_evt.bd_addr == XX:XX:XX:XX:XX:XX
```

Dovresti vedere un **indirizzo stabile** in tutti i pacchetti — questo è il PMAC che il Mac ha risolto grazie all'IRK.

Step 3 — Identifica il payload fingerprint

Cerca i pacchetti di advertising (ADV_IND o ADV_NONCONN_IND). Espandi un pacchetto BLE in Wireshark:

Cos'è `bthci_evt.code`: quando il controller BLE (il chip hardware) deve comunicare qualcosa all'host (il sistema operativo), lo fa tramite dei messaggi HCI chiamati HCI Events. Ogni evento ha un Event Code di 1 byte che identifica il tipo di evento. Il campo `bthci_evt.code` in Wireshark è proprio questo byte. Cosa significa `0x3e`: `0x3e` in decimale è 62, e corrisponde all'evento HCI chiamato LE Meta Event. Il LE Meta Event è un evento "contenitore" speciale: è l'unico canale attraverso cui il controller BLE comunica all'host tutti gli eventi specifici di Bluetooth Low Energy. Al suo interno c'è un secondo byte, il subevent code, che specifica cosa sta succedendo esattamente. Per esempio: `0x02` - LE Advertising Report (il dispositivo ha ricevuto un advertising packet); `0x01` - LE Connection Complete (è stata stabilita una connessione); `0x03` - LE Connection Update Complete; `0x05` - LE Long Term Key Request.

Bluetooth → Bluetooth HCI Event → LE Meta → Advertising Report
→ AD Structure → Entry Data

Cerca il campo **Manufacturer Specific Data** (tipo `0xFF`). Il payload Apple delle AirPods contiene dati come lo stato della batteria, il modello, ecc. — una parte di questo payload è **stabile** tra un RPA e l'altro.

Annota il valore hex del payload, ad esempio:

07:19:01:19:20:35:AA:B7:39:00:...

Crea il filtro Wireshark (click destro sul campo → "Apply as Filter"):

`btcommon.eir_ad.entry.data == 07:19:01:19:20:35:AA:B7:39:00:...`

Step 4 — I/O Graph (Alice)

In Wireshark: `Statistics → I/O Graph`

- Aggiungi un filtro per il PMAC: `bthci_evt.bd_addr == <PMAC>`
- Interval: 1 second
- Dovresti vedere una **linea continua e stabile** — un solo indirizzo per tutto il tempo

Fase 2 - Traccia TRUDY (osservatore passivo, nessun pairing)

Scenario

Trudy non ha l'IRK → non può risolvere gli RPA. Usa lo stesso hardware ma **rimuove il pairing** per simulare un osservatore esterno. In alternativa, usa l'**Android** con HCI snoop log (preferibile, perché fisicamente è un dispositivo diverso).

Opzione A — Mac come Trudy (rimuovi temporaneamente il pairing)

1. Vai in `Impostazioni di Sistema → Bluetooth`
2. Clicca le `...` accanto alle AirPods Pro → **Dimentica questo dispositivo**
3. Avvia PacketLogger e fai **Start**
4. Metti le AirPods vicino al Mac (ma NON fare il pairing)
5. Cattura per almeno **30-40 minuti** per osservare almeno una rotazione RPA
6. Salva come `Sxxxxxx_AirPodsPro2_Trudy.btsnoop`

Importante: Dopo aver salvato la traccia, riaccoppiate le AirPods normalmente.

Opzione B — Android come Trudy (consigliata, più pulita)

L'Android non ha mai avuto il pairing con le AirPods → è un Trudy naturale.

1. Abilita Developer Mode:

Impostazioni → Info sul telefono → Numero build (tappa 7 volte)

2. Abilita HCI snoop log:

Impostazioni → Opzioni sviluppatore → Abilita Bluetooth HCI snoop log

3. Attiva il Bluetooth sull'Android e **non fare pairing** con le AirPods

4. Tieni l'Android vicino alle AirPods per **30-40 minuti** (con AirPods che fanno advertising)

5. Trasferisci il file al Mac:

- Collega Android al Mac via USB
- Il file si trova in Internal Storage/ come btsnoop_hci.log
- Oppure via ADB: adb pull /sdcard/btsnoop_hci.log

6. Salva come Sxxxxxx_AirPodsPro2_Trudy.btsnoop

7. Disabilita il logging al termine:

BT off → disabilita l'opzione HCI snoop log

2.1 Analisi in Wireshark - Traccia Trudy

Step 1 — Rimuovi il rumore

```
!(bthci_evt.code == 0x3e)
```

Step 2 — Cerca il payload fingerprint trovato nella traccia Alice

```
btcommon.eir_ad.entry.data == 07:19:01:19:20:35:AA:B7:39:00:...
```

Tutti i pacchetti che compaiono appartengono allo stesso dispositivo fisico (le tue AirPods), ma con **MAC address diversi**.

Step 3 — Identifica gli RPA

Dai pacchetti filtrati, annota tutti gli indirizzi sorgente trovati:

```
bthci_evt.bd_addr
```

Ogni indirizzo distinto è un RPA diverso. Dovresti vederne almeno 2-3 in 30 minuti.

Step 4 — Conferma i singoli RPA

Per ogni RPA trovato, filtra i suoi pacchetti:

```
bthci_evt.bd_addr == AA:BB:CC:DD:EE:FF
```

Verifica che anche questi pacchetti abbiano il payload fingerprint → stesso dispositivo, indirizzo diverso.

Step 5 — I/O Graph (Trudy)

In Wireshark: `Statistics → I/O Graph`

- Aggiungi **una riga per ogni RPA** con colore diverso:
 - `bthci_evt.bd_addr == <RPA1>` → verde
 - `bthci_evt.bd_addr == <RPA2>` → rosso
 - `bthci_evt.bd_addr == <RPA3>` → blu
 - Interval: 1 second
 - Vedrai che le linee **non si sovrappongono nel tempo**: prima RPA1 è attivo, poi RPA2, poi RPA3 → la rotazione è visibile graficamente
-

Fase 3 - Linkability Attack

Obiettivo

Dimostrare che **nonostante la rotazione RPA**, un attaccante passivo può **collegare i diversi RPA allo stesso dispositivo fisico** usando il payload come fingerprint.

Algoritmo

Dalla traccia Alice:

1. Identifica il PMAC delle AirPods
2. Filtra tutti i suoi pacchetti: `bthci_evt.bd_addr == <PMAC>`
3. Individua un payload stabile e specifico (Manufacturer Specific Data)
4. Costruisci il filtro payload: `btcommon.eir_ad.entry.data == <payload>`

Nella traccia Trudy:

5. Applica lo stesso filtro payload
6. Osserva i MAC address dei pacchetti risultanti → sono gli RPA
7. Per ogni RPA trovato: `bthci_evt.bd_addr == <RPA_n>`
8. Verifica consistenza: stesso payload, stesso intervallo di advertising, RSSI simile
9. Costruisci la mappa: PMAC → RPA1, RPA2, RPA3, ...

Risposta alla domanda del lab

■ *"Can payload-level fingerprints deanonymize devices?"*

Sì, almeno parzialmente. Le AirPods Pro 2 includono nel payload BLE dati Apple-specifici (tipo dispositivo, stato batteria, ecc.) che non cambiano con la rotazione RPA. Questo rende possibile la

correlazione cross-RPA anche senza l'IRK. La protezione RPA è quindi **necessaria ma non sufficiente**: la privacy dipende anche dall'opacità del payload.

Fase 4 - Domanda bonus del lab

└─ "Can you see the Secure Connections messages? Why?"

Le AirPods Pro 2 usano **BLE Secure Connections** (BT 4.2+). In Wireshark, filtra:

```
_ws.col.protocol == "SMP"
```

Probabilmente **non vedrai i messaggi SMP di pairing** (Public Key exchange, Confirm, Random, DHKey Check). Questo perché:

1. Wireshark cattura a livello HCI — i messaggi SMP *ci sono*, ma la fase di pairing avviene raramente (solo al primo accoppiamento o dopo aver dimenticato il dispositivo)
 2. Se vedi messaggi SMP, cerca: `Opcode: Pairing Public Key (0x0c)` per lo scambio ECDH
 3. I dati applicativi post-pairing sono **cifrati con AES-CCM** → appaiono come payload opaco in Wireshark
-

Consegna finale

Struttura del file .zip

```
Sxxxxxx_BTLAB.zip
├─ Sxxxxxx_AirPodsPro2_Alice.btsnoop  (o .pcap/.pcapng)
├─ Sxxxxxx_AirPodsPro2_Trudy.btsnoop
└─ Setup.txt
```

Contenuto di Setup.txt

```
Device:      Apple AirPods Pro 2 (generazione 2)
Vendor:      Apple Inc.
OS:          macOS Sequoia 15.x (MacBook Air M2)
Tools:       PacketLogger (Xcode Additional Tools 16), Wireshark 4.x
BT Adapter:  Integrated Apple BCM (MacBook Air M2)
```

```
Trudy setup: [Android / Mac con pairing rimosso]
```

```
=== RPA identificati nella traccia Trudy ===
```

```
RPA1: AA:BB:CC:DD:EE:FF  (attivo da T=0 a T=900s circa)
```

```
RPA2: 11:22:33:44:55:66  (attivo da T=900 a T=1800s circa)
```

RPA3: ...

```
=== Payload fingerprint usato per linkability ===  
btcommon.eir_ad.entry.data == 07:19:01:19:20:35:...
```

=== Wireshark filters principali ===

Alice PMAC: bthci_evt.bd_addr == XX:XX:XX:XX:XX:XX

Trudy RPA1: bthci_evt.bd_addr == AA:BB:CC:DD:EE:FF

Payload FP: btcommon.eir_ad.entry.data == <hex>

No adv noise: !(bthci_evt.code == 0x3e)

Upload

<https://www.dropbox.com/request/drl4jkiehro4e7hs05js>

Tabella riassuntiva dei filtri Wireshark

Scopo	Filtro
Filtra per indirizzo MAC	<code>bthci_evt.bd_addr == XX:XX:XX:XX:XX:XX</code>
Filtra per protocollo	<code>_ws.col.protocol == "SMP"</code>
Rimuovi advertising report	<code>!(bthci_evt.code == 0x3e)</code>
Filtra per payload specifico	<code>btcommon.eir_ad.entry.data == AA:BB:CC:...</code>
Solo pacchetti BLE	<code>bthci_evt.le_meta_subevent == 0x02</code>
Solo advertising events	<code>bthci_evt.code == 0x3e</code>

Note e troubleshooting

PacketLogger non cattura nulla → Verifica che il logging profile sia installato e fidato in

`Impostazioni di Sistema → Generali → Gestione VPN e dispositivi`

Non vedo advertising delle AirPods → Le AirPods fanno advertising solo quando non sono in connessione attiva con nessun dispositivo, oppure quando sono nella custodia con il coperchio aperto. Disconnettile dal Mac prima di catturare la traccia Trudy.

Non trovo il payload fingerprint → Prova a cercare nei campi `btcommon.eir_ad.entry.type == 0xff` (Manufacturer Specific) o `btcommon.eir_ad.entry.type == 0x16` (Service Data). Le AirPods usano entrambi.

La rotazione RPA non avviene in 15 minuti → Apple può usare intervalli diversi. Cattura per almeno 30-45 minuti per essere sicuro di osservare almeno un cambio. Puoi forzarlo mettendo le AirPods in airplane mode e riattivandole.